



`$state.frozen` / `$state.raw`: Opting Out of Deep Reactivity

Q36

SVELTE 5 RUNES `$STATE.RAW` PERFORMANCE

INTERVIEW QUESTION

Question. What is the `$state.frozen` rune, and in what scenarios should it be chosen over standard `$state`?

ANSWER. The rune that was prototyped as `$state.frozen` shipped in Svelte 5 as `**$state.raw*`. *It declares reactive state that Svelte does not* wrap in a deep Proxy: assignment to the variable itself is reactive (`config = newConfig` triggers updates), but mutating a nested property (`config.theme = 'dark'`) is silent — there is no proxy to detect it, so no update fires.

Use it whenever you hold **read-mostly structured data that you replace wholesale instead of mutate in place**: configuration objects, look-up tables, parsed fixtures, feature flags, immutable shapes received from a store. The payoff is concrete: Svelte skips the per-property proxy allocation on every access, so cold reads and initial mount of large objects are cheaper, and you stop paying for fine-grained tracking you never use.

The trap is symmetric: reaching for `$state.raw` to “optimize” a value you then mutate at depth is the same bug as legacy Svelte’s `user.age = 31` — the change lands in memory but the UI never hears about it. The rule is prescriptive, not stylistic: deep reactivity is the default because it is always correct; `$state.raw` is an opt-out you take only when you commit to the replace-don’t-mutate discipline for that value.

```
<script>
  // Deeply reactive: every nested property is tracked.
  // Use when you will mutate fields in place.
```

App.svelte

```

let session = $state({ user: { name: 'Ada', role: 'admin' } });

// Not deeply reactive: assignment is reactive, mutation is not.
// Use when you always replace the whole value.
let config = $state.raw({
  apiUrl: 'https://api.example.com',
  retries: 3,
  features: { beta: false },
});

function patchConfig(next) {
  // ✅ Replace the reference → UI updates.
  config = { ...config, ...next };
}

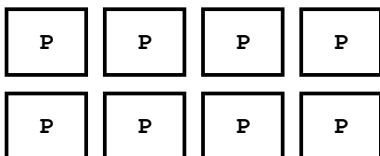
function misuseConfig() {
  // ❌ Silent: no proxy here, so this change never reaches the
  DOM.
  config.features.beta = true;
}
</script>

<p>{session.user.name} ({session.user.role})</p>
<p>API: {config.apiUrl}, retries: {config.retries}</p>
<button onClick={() => patchConfig({ retries: 5 })}>Bump
retries</button>

```

❶ **Summary.** `$state.raw` (the rune prototyped as `$state.frozen`) is the deliberate off-switch for deep proxying. Choose it for read-mostly structured data that you replace rather than mutate, and leave `$state` as the default everywhere else — because only the deep variant is correct when you need in-place edits to render.

`$STATE (HUGEARRAY)`



PROXY ON EVERY PROPERTY

`$STATE . RAW (HUGEARRAY)`

[...one
value...]

TRACKED ONLY AT THE
BINDING